S: CogDL (Software)
T: NU Embed, Node Class, Graph Class, Heterogeneous Node class, Link Pred, KG Completion, Graph Clustering, Graph Attack + Defend, Multiplex link Pred, Top-K Sim Search
D: Facebook
P: Under paper presents CogDL as a framework to implement GNN models more efficiently.
M: Speedup (ET)
Various effect Metrics for each Dataset + Task.

Check for updates

CogDL: A Comprehensive Library for Graph Deep Learning

Yukuo Cen, cyk20@mails.tsinghua.edu.cn
Zhenyu Hou, houzy21@mails.tsinghua.edu.cn
Yan Wang, wang-y17@mails.tsinghua.edu.cn
Tsinghua University, China

Qibin Chen, cqb19@mails.tsinghua.edu.cn
Yizhen Luo, luoyz18@mails.tsinghua.edu.cn
Zhongming Yu, yzm18@mails.tsinghua.edu.cn
Tsinghua University, China

Hengrui Zhang, hengrui-18@mails.tsinghua.edu.cn
Xingcheng Yao, yxc18@mails.tsinghua.edu.cn
Aohan Zeng, zah22@mails.tsinghua.edu.cn
Tsinghua University, China

Shiguang Guo, Tsinghua University, China, gsg18@mails.tsinghua.edu.cn
Yuxiao Dong, Tsinghua University, China, yuxiaod@tsinghua.edu.cn
Yang Yang, Zhejiang University, China, yangya@zju.edu.cn
Peng Zhang, Zhipu AI, China, zpjumper@gmail.com
Guohao Dai, Shanghai Jiao Tong University, China, daiguohao@sjtu.edu.cn
Yu Wang, Tsinghua University, China, yu-wang@tsinghua.edu.cn
Chang Zhou, Alibaba Group, China, ericzhou.zc@alibaba-inc.com
Hongxia Yang, Alibaba Group, China, yang.yhx@alibaba-inc.com
Jie Tang*, Tsinghua University, China, jietang@tsinghua.edu.cn

ABSTRACT

Graph neural networks (GNNs) have attracted tremendous attention from the graph learning community in recent years. It has been widely adopted in various real-world applications from diverse domains, such as social networks and biological graphs. The research and applications of graph deep learning present new challenges, including the sparse nature of graph data, complicated training of GNNs, and non-standard evaluation of graph tasks. To tackle the issues, we present CogDL[1], a ***co***mprehensive library for ***g***raph ***d***eep ***l***earning that allows researchers and practitioners to conduct experiments, compare methods, and build applications with ease and efficiency. In CogDL, we propose a unified design for the training and evaluation of GNN models for various graph tasks, making it unique among existing graph learning libraries. By utilizing this unified trainer, CogDL can optimize the GNN training loop with several training techniques, such as mixed precision training. Moreover, we develop efficient sparse operators for CogDL, enabling it to become the most competitive graph library for efficiency. Another important CogDL feature is its focus on ease of use with the aim of facilitating open and reproducible research of graph learning. We leverage CogDL to report and maintain benchmark results on fundamental graph tasks, which can be reproduced and directly used by the community.

*Jie Tang is the corresponding author.
[1]The code is available at: https://github.com/THUDM/cogdl

This work is licensed under a Creative Commons Attribution International 4.0 License.

WWW '23, April 30–May 04, 2023, Austin, TX, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9416-1/23/04.
https://doi.org/10.1145/3543507.3583472

CCS CONCEPTS

• **Software and its engineering** → **Software libraries and repositories**; • **Mathematics of computing** → **Graph algorithms**; • **Computing methodologies** → *Learning latent representations.*

KEYWORDS

graph deep learning, graph neural networks, graph library.

ACM Reference Format:
Yukuo Cen, Zhenyu Hou, Yan Wang, Qibin Chen, Yizhen Luo, Zhongming Yu, Hengrui Zhang, Xingcheng Yao, Aohan Zeng, Shiguang Guo, Yuxiao Dong, Yang Yang, Peng Zhang, Guohao Dai, Yu Wang, Chang Zhou, Hongxia Yang, and Jie Tang. 2023. CogDL: A Comprehensive Library for Graph Deep Learning. In *Proceedings of the ACM Web Conference 2023 (WWW '23), April 30–May 04, 2023, Austin, TX, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3543507.3583472

1 INTRODUCTION

Graph-structured data have been widely utilized in many real-world scenarios. Inspired by recent trends of representation learning on computer vision (CV) and natural language processing (NLP), graph neural networks (GNNs) [34, 71] are proposed to apply neural architectures to perform message passing over graph structures. For example, graph convolutional networks (GCNs) [48] simplify the spectral graph convolutions with first-order approximation and utilize a layer-wise propagation rule of graph convolution. GraphSAGE [37] adopts a sampling-based method to generate node embeddings for large graphs, which can be applied to the inductive setting. Recent effort in GNNs has been focused on theoretical understanding [98], generalization [26], self-supervised learning [43, 86], and capacity to handle web-scale applications [104]. To date, GNNs have achieved impressive performance on various graph machine learning tasks in diverse domains [42].

With the prosperity of GNNs, the evaluation and fair comparison of GNN models can still be a concerning issue. Although there are enormous research works on graph machine learning, we observe that different papers may use their evaluation settings for the same graph dataset, making results indistinguishable. For example, as a widely used dataset, Cora [73], some papers use the "standard" semi-supervised splits following Planetoid [102], while others adopt random splits or fully-supervised splits. Reported results of a GNN model on the same dataset may differ in various papers, making it challenging to compare the performance of different GNN methods [22, 24, 42, 74].

Moreover, the graph-structured data is fundamentally different from natural language or images, which can be easily formatted as `Tensor` and thus straightforwardly processed by GPUs. The non-Euclidean and sparse nature of graph data requires a better storage format for efficient computation, especially on GPUs. However, current deep learning frameworks do not well support the sparse computation of graph-structure data. For example, the PyTorch API of sparse tensors is limited and inefficient for graph operations. To bridge the gap, several open-source efforts have been made to develop dedicated libraries for the efficient development of graph representation learning research. For example, PyTorch Geometric (PyG) [27] is a graph learning library built upon PyTorch [62] to easily write and train GNNs for various applications. Deep Graph Library (DGL) [91] is an efficient and scalable package for deep learning on graphs, which provides several APIs allowing arbitrary message-passing computation over large-scale graphs. However, these popular graph libraries mainly focus on implementing basic GNN operators and do not take the whole training and evaluation of GNNs into consideration. The end-to-end performance of training GNNs could still be improved.

Present work: CogDL. In this paper, we introduce CogDL, a ***co***mprehensive library for ***g***raph ***d***eep ***l***earning that allows researchers, practitioners, and developers to train and evaluate GNN models under different graph tasks with ease and efficiency. The advantage of CogDL lies in its design of a unified GNN trainer, standard evaluation, and modular wrappers. Specifically, CogDL unifies the training and evaluation of graph neural network (GNN) models, making it different from other graph learning libraries. In addition, it is also equipped with decoupled modules that can be flexibly plugged for training GNNs, including `ModelWrapper` and `DataWrapper`. CogDL provides built-in GNN-specific wrappers, which could not be offered by other libraries.

The goal of CogDL is to accelerate research and applications of graph deep learning. Based on the design of the unified trainer and decoupled modules, CogDL users can instantiate a GNN trainer by `trainer = Trainer(...)` and then call `trainer.run(...)` to conduct experiments easily, which is not supported by existing graph libraries. We also provide a high-level API (i.e., `experiment`) on top of `Trainer` for one-line usage. Using the unified trainer not only saves users' time in writing additional code but also gives users the opportunity to enable many built-in training techniques, such as mixed precision training. For example, users only need to set `Trainer(fp16=True)` to enable the feature of mixed precision training without any modification. Furthermore, CogDL provides as many tasks with standard evaluation as possible for users (as shown in Table 1), and maintains experimental results on these fundamental graph tasks with recorded hyperparameter configurations. We summarize the characteristics of CogDL as follows:

Listing 1: One-line code to conduct experiments.

```
from cogdl import experiment
experiment(dataset=["cora","citeseer"], model=["gcn","gat","gcnii"])
```

- **Ease of use**. We provide simple APIs in CogDL such that users only need to write one line of code to train and evaluate any graph representation learning methods. CogDL develops a unified trainer with decoupled modules for users to train GNNs easily. Based on this unique design, CogDL can provide extensive features such as hyperparameter optimization, efficient training techniques, and experiment management.
- **End-to-end performance**. Efficiency is one of the most significant characteristics of CogDL. CogDL develops well-optimized sparse kernel operators to speed up the training of GNN models. For example, these efficient sparse operators enable CogDL to achieve about 2× speedups on the 2-layer GCN [48] and GAT [87] models compared with PyG and DGL.
- **Reproducibility**. Users can quickly conduct the experiments to reproduce the results based on CogDL, as illustrated in Listing 1. CogDL implements 70+ models and collects 70+ datasets for 10 fundamental graph tasks, as shown in Table 1, facilitating open, robust, and reproducible deep learning research on graphs.

Relevance to Web. GNNs have been widely used in web research and applications, such as social influence prediction [64, 68], social recommendation [25], and web advertising [103]. However, there still remain several challenges to applying powerful GNN models to real web applications. Practitioners need to not only choose an appropriate GNN but also train the GNNs on large-scale graphs. To mitigate the issues, CogDL provides unique advantages for web research and applications. First, the well-optimized sparse operators can speed up the process of new research and applications. Besides, the unified trainer with built-in `wrappers` can be directly utilized to conduct experiments with ease and efficiency. In addition, CogDL provides the benchmarking results of GNN methods for all kinds of graph tasks, which could help web-related users to choose appropriate methods. Finally, we have successfully applied CogDL to a real-world academic system—AMiner. More details about the application can be found in Appendix B.

2 THE COGDL FRAMEWORK

In this section, we present the unified design of the CogDL library, which is built on PyTorch [62], a well-known deep learning library. Based on PyTorch, CogDL provides efficient implementations of many models and our designed sparse operators. To date, it supports 10 important graph tasks, such as node classification. Table 1 lists all supported tasks, datasets, and models in the CogDL. All the models and datasets are compatible with the unified trainer for conducting experiments on different graph tasks.

Table 1: All supported tasks, datasets, and models in CogDL. To date, CogDL integrates 10 important graph tasks with the standard evaluation of 70+ datasets. Besides, CogDL implements 70+ graph representation methods, which can be directly used by users. Note that some datasets and methods can be used in several tasks.

Tasks	Datasets	Models
Network Embedding	PPI, Wikipedia, Blogcatalog, DBLP, Flickr, YouTube	DeepWalk [63], LINE [80], node2vec [36], ProNE [110], NetMF [67], NetSMF [66], SDNE [90], Spectral [83], DNGR [12], GraRep [11], HOPE [61], AutoProNE [40]
Node Classification	Cora, Citeseer, Pubmed, Chameleon, Cornell, Film, Squirrel, Texas, Wisconsin, PPI, Flickr, Reddit, Amazon, Yelp, GitHub, Elliptic, Yelpchi, Alpha, Clothing, Electronics, Wiki, Weibo, BGP, SSN, ogbn-arxiv, ogbn-products, ogbn-proteins, ogbn-papers100M,	Chebyshev [20], GCN [48], GAT [87], GraphSAGE [37], APPNP [49], DGI [86], GCNII [17], MVGRL [39], GRAND [26], DropEdge [69], Graph-Unet [31], GDC [50], PPRGo [8], GraphSAINT [109], GraphMix [88], SIGN [28], MixHop [3], GRACE [117], DeeperGCN [53], DisenGCN [56], SGC [96], M3S [77], C&S [46], SAGN [75], RevGNN [52], Cluster-GCN [18], InfoMotif [70], GraphMAE [41]
Graph Classification	MUTAG, PTC, NCI1, PROTEINS, IMDB-B, IMDB-M, COLLAB, REDDIT-B, REDDIT-M5k, REDDIT-M12k, NCI09, ENZYMES, ogbg-molhiv, ogbg-molpcba, ogbg-ppa, ogbg-code	DGK [100], graph2vec [58], GIN [98], PATCHY_SAN [60], DiffPool [105], InfoGraph [76], DGCNN [94], SortPool [111]
Heterogeneous Node Classification	DBLP, ACM, IMDB	Simple-HGN [55], GTN [107], HAN [92], PTE [79], Metapath2vec [21], Hin2vec [29]
Link Prediction	Cora, Citeseer, Pubmed, ogbl-ppa, ogbl-ddi, ogbl-collab, ogbl-citation2	GAE [47], VGAE [47], network embedding methods
Knowledge Graph Completion	FB13, FB15k, FB15k-237, WN18, WN18RR	TransE [9], DistMult [101], ComplEx [84], RotatE [78], RGCN [72], CompGCN [85]
Graph Clustering	Cora, Citeseer, Pubmed	GAE [47], VGAE [47], AGC [112], DAEGC [89]
Graph Attacks and Defenses	grb-cora, grb-citeseer, grb-flickr, grb-reddit, grb-aminer	GNN-SVD [23], GNNGuard [113], RobustGCN [115], RND [119], TDGIA [118], FGSM [33], PGD [57], SPEIT [114], DICE [95], FGA [15], FLIP [7], NEA [7], STACK [97]
Multiplex Link Prediction	Amazon, YouTube, Twitter	GATNE [13], network embedding methods
Top-k Similarity Search	KDD-ICDM, SIGIR-CIKM, SIGMOD-ICDE	GCC [65]

2.1 Unified Trainer

CogDL provides a unified trainer for GNN models, which takes over the entire loop of the training process. The unified trainer, which contains much engineering code, is implemented flexibly to cover arbitrary GNN training settings. We design four decoupled modules for the GNN training, including `Model`, `ModelWrapper`, `Dataset`, `DataWrapper`. The `ModelWrapper` is for the training and testing steps, while the `DataWrapper` is designed to construct data loaders used by `ModelWrapper`. The main contributions of most GNN papers mainly lie in three modules except `Dataset`. For example, the GCN paper trains the GCN model under the (semi-)supervised and full-graph setting, while the DGI paper trains the GCN model by maximizing local-global mutual information. The training method of the DGI is considered as a model wrapper named `dgi_mw`, which could be used for other scenarios. Our unified design also supports the precomputation-based GNN models, such as SIGN [28] and SAGN [75].

Based on the design of the unified trainer and decoupled modules, we could do arbitrary combinations of models, model wrappers, and data wrappers. For example, if we want to apply the DGI, a representative graph self-supervised learning method, to large-scale datasets, all we need is to substitute the full-graph data wrapper with the neighbor-sampling or clustering data wrappers without additional modifications. If we design a new GNN architecture for a specific task, we only need to write essential PyTorch-style code for the model. The rest could be automatically handled by CogDL by specifying the model wrapper and the data wrapper. We could quickly conduct experiments for the model using the trainer via

`trainer = Trainer(epochs,...)` and `trainer.run(...)`. Moreover, based on the unified trainer, CogDL provides native support for many useful features, including hyperparameter optimization, efficient training techniques, and experiment management without any modification to the model implementation.

We introduce efficient techniques of GNN training, which credit to the unified trainer of CogDL. These features could be enabled through `fp16=True` or `actnn=True` in `Trainer` API.

Mixed Precision Training. CogDL supports mixed precision training, which is a popular technique to relieve the GPU memory and speed up the training process. PyTorch provides a convenient method for mixed precision in `torch.cuda.amp`, which integrates NVIDIA apex. For example, we conduct experiments to test the performance of mixed precision training.

Activation Compression Training. With the emergence of deep GNNs, the activation footprints occupy more and more GPU memory. And we need to store the activation output of each layer to compute the gradient in the backward pass, which costs much GPU memory in the training step. We extend the activation compression training technique [16, 54] to GNNs. The main advantage of activation compression training is that the GPU memory of training could dramatically decrease.

Based on the provided trainer, CogDL also supports the following features for conducting experiments with ease.

Hyperparameter Optimization. Hyperparameter optimization (HPO) is an important feature for GNN libraries since current GNN models utilize more hyperparameters than before. We integrate a popular library, Optuna [4], into CogDL to enable HPO. Optuna is a fast AutoML library based on Bayesian optimization. CogDL implements hyperparameter search and even model search for graph learning. The key of the HPO is to define a search space. The search space will be automatically utilized by CogDL to start searching and output the best results. The usage of HPO can be found in Appendix A.2.

Experiment Management. Experiment management is crucial for training deep learning models, which could be utilized by researchers and developers for debugging and designing new blocks. CogDL integrates TensorBoard and WandB [6] for experiment management. TensorBoard is a common visualization toolkit for tracking metrics like loss and accuracy. WandB is a central dashboard to keep track of the experimental status. The experiment management could be enabled easily by assigning the `logger` argument.

2.2 Efficient Sparse Operators

We introduce the well-optimized sparse operators of CogDL, including SpMM-like operators, multi-head SpMM, and edge-wise softmax, which are developed and implemented in CogDL for GNN models. We first introduce the abstraction of sparse operators in CogDL and its corresponding optimization. The General Sparse Matrix-Matrix multiplication (*GSpMM*) operator is widely used in most GNNs. The reason is that many GNNs apply a general aggregation operation for each node from its neighbors:

$$\boldsymbol{h}_u^{(l+1)} = \phi\left(\left\{\psi\left(\boldsymbol{h}_v^{(l)}, \boldsymbol{m}_e\right) \mid (u,e,v) \in E\right\}\right) \tag{1}$$

where (u,e,v) denotes an edge e from node u to node v with a message $\boldsymbol{m}_e$, and $\boldsymbol{h}_u^{(l)}$ is the hidden representation of node u of layer l. When the reduce operation ϕ is a summation, and the compute operation ψ is a multiplication, such an operation can be described as an SpMM operator $\boldsymbol{H}^{(l+1)} \leftarrow \boldsymbol{A}\boldsymbol{H}^{(l)}$, where the sparse matrix $\boldsymbol{A}$ represents the adjacency matrix of the input graph G, and each row of $\boldsymbol{H}^{(l)}$ represents a representation vector of nodes (e.g., $\boldsymbol{h}_u^{(l)}$ and $\boldsymbol{h}_v^{(l)}$).

We observe that in existing graph neural network libraries such as PyG [27], or DGL [91], their backend support is not tailored to GNN. In PyG, the message-passing flow is built by torch-scatter, which is borrowed from the design principles of traditional graph processing. In DGL, its GSpMM function leverage APIs from cuSPARSE [59], which is a vendor library that mainly targets for scientific computing of sparse matrices. Another observation is that many previous GPU-based SpMM works fail to be aware of the adaptability of workload balance in graphs. For example, GE-SpMM [44] and Sputnik [30] are row-split SpMM methods that could not efficiently support graphs that follow a power-law distribution. CogDL is fully aware of the work-load balance problem compared to PyG/DGL. We observe that the algorithm of neighbor group [45, 93] could better slice the large nodes in the graph and thus has better efficiency when facing power-law distribution graphs. However, some graphs may naturally conform to node load balancing, such as the k-sampled graph [37], so it is better to use the row-split backend directly.

To tackle the following problems, we propose a new balance abstraction that not only exploits the data of the graph itself but also effectively accelerates our backend. As shown in Listing 2, our workload balance handler would be generated by a data-aware balancer, which could indicate whether to use a row-split or neighbor group algorithm. Empirically, when the degrees of the majority (90%) of the nodes are close to the average degree, then we would choose row-split as our backend since it is suitable for graphs that are naturally node balanced. Note that our workload balancer is a preprocessing method since it would generate the auxiliary arrays that are needed by the GSpMM backend. The Listing 2 demonstrates that our GSpMM's inputs include a work balance handler, graph data, and input features. Users could choose the reduce and compute operator, as illustrated in Equation 1.

Listing 2: workload balance abstraction in CogDL backend

```
workload_balance = balancer(graph) # preprocessing
out_feat = GSpMM(workload_balance, graph, in_feat, reduce_op,
      compute_op)
```

In our GSpMM design, we implement the warp-based thread mapping algorithm, which would bind different neighbor groups or split rows to different warps. This allows us to assign multiple adjacent rows or neighbor groups into a single CUDA block. Since the same processor shares an L1 cache, adjacent rows or neighbor groups can reduce the L1 loaded data to improve locality. Multi-head SpMM is used to compute the aggregation procedure of GNN models with multi-head attention mechanisms (such as GAT), which means each edge has a different edge weight for each attention head; therefore, multiple SpMM operators are needed. PyG and DGL use multiple SpMM with broadcasting to implement it. In

Table 2: Performance of mixed precision training.

	Memory	Accuracy	Training Speed	
			2080 Ti	3090
w/o fp16	5,567 MB	95.44	2.20 it/s	3.93 it/s
with fp16	4,046 MB	95.35	3.17 it/s	7.97 it/s

Table 3: Accuracy (%) and activation memory (MB) of 2-layer GCNs on three datasets.

	origin (32bit)	act (4bit)	act (3bit)	act (2bit)
Flickr	51.17 (288)	51.08 (37)	51.14 (26)	51.20 (18)
Reddit	95.33 (1532)	95.32 (194)	95.31 (158)	95.34 (112)
Yelp	39.86 (4963)	40.06 (773)	40.21 (665)	39.89 (551)

our multi-head SpMM kernel, we apply the memory coalescing technique by allocating consecutive warps along the dimension of the head. As different heads share the same location of non-zero elements, this data will be cached by the streaming multiprocessors and reused among different heads, which also saves bandwidth.

When considering a GAT model, we also need a sampled dense-dense matrix multiplication (SDDMM) operator $\boldsymbol{T} = \boldsymbol{A} \odot (\boldsymbol{P}\boldsymbol{Q}^T)$, where $\boldsymbol{T}$ and $\boldsymbol{A}$ are the sparse matrices, $\boldsymbol{P}$ and $\boldsymbol{Q}$ are two dense matrices, $\odot$ is element-wise matrix multiplication. The SDDMM operator is used for back-propagating the gradients to the sparse adjacency matrix since the adjacency matrix of the GAT model is computed by the attention mechanism. CogDL further speeds up multi-head graph attention by optimizing Edge-wise Softmax, which is defined by $\boldsymbol{\alpha}'_{uv} = \exp(\boldsymbol{\alpha}_{uv})/(\sum_{v\in\mathcal{N}(u)} \exp(\boldsymbol{\alpha}_{uv}))$, where $\boldsymbol{\alpha}_{uv}$ is the attention score between nodes u and v.

In our design of the Edge-wise Softmax kernel, we use the method of warp level intrinsic provided by NVIDIA, a typical method that could accelerate the scan and reduce operation. To prevent the spillover of the exponent, we first apply the scan to find the max value of edge weight and then subtract this maximum from each value. After that, we compute each value by applying the exponent function and reduce all the values within the warp to acquire the sum of all the exponent values. As far as we know, CogDL is the first framework that provides a GPU-based fused Edge-wise Softmax kernel. Therefore, we do not provide the results of the experiment with PyG and DGL.

3 EVALUATION

In this section, we conduct extensive experiments to evaluate the framework design and characteristics of CogDL.

3.1 Training Techniques

For mixed precision training, we run a 2-layer GCN model with 2048 hidden units on the Reddit dataset. From Table 2, the mixed precision training brings 27% memory savings and 1.44× ∼ 2.02× speedup on NVIDIA 2080 Ti and 3090 GPUs with almost no performance drop. For activation compression training, we conduct experiments of the 2-layer GCN model with 256 hidden units on three datasets for the activation compression training in the CogDL, as shown in Table 3. The performance demonstrates that training GNNs with actnn brings 6.4× ∼ 16× memory savings compared with 32-bit training. The accuracy with actnn (2,3,4-bit) is almost the same with 32-bit training.

Table 4: End-to-end inference time in seconds of 2-layer GCN and GAT models with hidden size 128. The GAT model uses 4 attention heads. OOM means out of memory. PyG will be more likely to occur out of memory because it is using the torch-scatter method that will store the temporary edge embedding in the GPU memory.

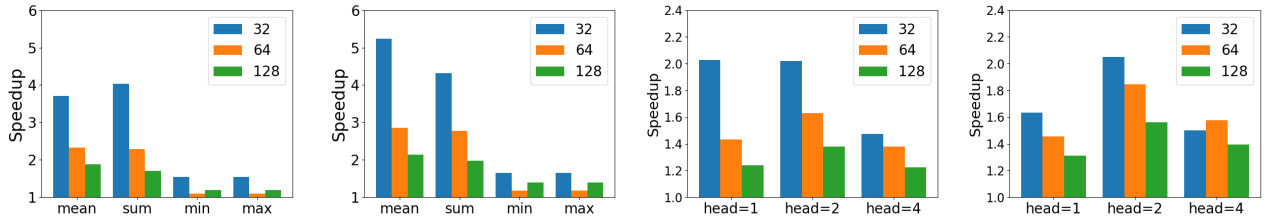
Model	GPU	Dataset	PyG	DGL	CogDL
GCN	2080Ti (11G)	Flickr	0.004	0.007	0.004
		Reddit	0.045	0.049	0.039
		Yelp	0.053	0.063	0.042
	3090 (24G)	Flickr	0.002	0.004	0.002
		Reddit	0.023	0.031	0.022
		Yelp	0.029	0.036	0.023
GAT	2080Ti (11G)	Flickr	0.033	0.019	0.014
		Reddit	OOM	0.214	0.131
		Yelp	OOM	OOM	0.138
	3090 (24G)	Flickr	0.021	0.013	0.009
		Reddit	OOM	0.112	0.080
		Yelp	OOM	0.107	0.081

3.2 Sparse Operators

We conduct experiments of GSpMM and multi-head SpMM on Reddit and Yelp datasets compared to DGL, as shown in Figure 1. For GSpMM, CogDL achieves 1.17× ∼ 3.47× speedup with different reduce functions on Reddit and 1.27× ∼ 4.01× speedup with different reduce functions on Yelp. For multi-head SpMM, CogDL achieves 1.22× ∼ 2.03× and 1.31× ∼ 2.05× speedup on Reddit and Yelp datasets, respectively.

3.3 End-to-End Performance

We compare the end-to-end inference time of GCN and GAT models on several datasets with other popular GNN frameworks: CogDL v0.5.2, PyTorch-Geometric (PyG) v2.0.2, and Deep Graph Library (DGL) v0.7.2 with PyTorch backend. The statistics of datasets could be found in Table 9. We conduct experiments using Python 3.7.10 and PyTorch v1.8.0 on servers with Nvidia GeForce RTX 2080 Ti (11GB GPU Memory) and 3090 (24GB GPU Memory). From Table 4, CogDL achieves at most 2× speedup on the 2-layer GCN model compared with PyG and DGL. For the 2-layer GAT model, CogDL achieves 1.32× ∼ 2.36× speedup compared with PyG and DGL. Furthermore, OOM occurs when running the PyG's GAT model on Reddit and Yelp datasets, even using 3090 (24G). The GAT model implemented by DGL also cannot run the Yelp dataset using 2080Ti (11G). The results demonstrate significant advantages of CogDL in inference time and memory savings, compared to state-of-the-art GNN frameworks.



(a) GSpMM on Reddit (b) GSpMM on Yelp (c) multi-head SpMM on Reddit (d) multi-head SpMM on Yelp

Figure 1: Speedup of GSpMM and multi-head SpMM with 32, 64, 128 hidden units compared with DGL. (a) $1.70\times \sim 4.04\times$ **speedup with mean and sum as reduce functions on Reddit. (b)** $1.98\times \sim 5.24\times$ **speedup with mean and sum as reduce functions on Yelp. (c)** $1.22\times \sim 2.03\times$ **speedup with 1, 2, 4 attention heads on Reddit. (d)** $1.31\times \sim 2.05\times$ **speedup with 1, 2, 4 attention heads on Yelp.**

4 BENCHMARKING

In this section, with CogDL, we introduce several downstream tasks to evaluate implemented methods. The statistics of the datasets are shown in Table 9 and Table 10 in the Appendix. Based on the `experiment` API, we build a reliable leaderboard for each task, which maintain state-of-the-art results.

4.1 Network Embedding

Setup. Unsupervised node classification task aims to learn a mapping function $f : V \mapsto \mathbb{R}^d$ that projects each node to a d-dimensional space ($d \ll |V|$) in an unsupervised manner. Structural properties of the network should be captured by the mapping function. We collect the most popular datasets used in the unsupervised node classification task, including BlogCatalog [108], Wikipedia [67], PPI [10], DBLP [81], Flickr [82]. We implement and compare the following methods for the unsupervised node classification task, including SpectralClustering [83], DeepWalk [63], LINE [80], node2vec [36], GraRep [11], HOPE [61], NetMF [67], ProNE [110], NetSMF [66]. Skip-gram network embedding considers the vertex paths traversed by random walks over the network as the sentences and leverages Skip-gram for learning latent vertex representation. For matrix factorization based methods, they first compute a proximity matrix and perform matrix factorization to obtain the embedding. Actually, NetMF [67] has shown the aforementioned Skip-gram models with negative sampling can be unified into the matrix factorization framework with closed forms. We run all network embedding methods on five real-world datasets and report the Micro-F1 results (%) using logistic regression with L2 normalization.

Results and Analysis. Table 5 shows the results and we find some interesting observations. DeepWalk and node2vec consider vertex paths traversed by random walk to reach high-order neighbors. NetMF and NetSMF factorize diffusion matrix $\sum_{i=0}^{K} \alpha_i \boldsymbol{A}^i$ rather than adjacency matrix $\boldsymbol{A}$. ProNE and LINE are essentially 1-order methods, but ProNE further propagates the embeddings to enlarge the receptive field. Incorporating global information can improve performance but may hurt efficiency. The propagation in ProNE, which is similar to graph convolution, shows that incorporating global information as a post-operation is effective. The results demonstrate that matrix factorization (MF) methods like NetMF and ProNE are very powerful and full of vitality as they outperform Skip-gram based methods in almost all datasets. ProNE and NetSMF are also of high efficiency and scalability and able to embed super-large graphs in feasible time in one single machine.

4.2 Node Classification

Setup. This task is for node classification with GNNs in semi-supervised and self-supervised settings. Different from the previous part, nodes in these graphs, like Cora and Reddit, have node features and are fed into GNNs with prediction or representation as output. Cross-entropy loss and contrastive loss are set for semi-supervised and self-supervised settings, respectively. The datasets consist of two parts, including both semi-supervised and fully-supervised settings. Semi-supervised datasets include three citation networks, Citeseer, Cora, and Pubmed [73]. Fully-supervised datasets include social networks (Reddit, Yelp, and Flickr), bioinformatics (PPI) from GraphSAINT [109], and ogbn-arxiv [42]. We implement all the following models, including ChebyNet [20], GCN [48], GAT [87], GraphSAGE [37], APPNP [49], DGI [86], GCNII [17], MVGRL [39], GRAND [26], DropEdge [69], Graph-Unet [31], GDC [50], PPRGo [8], GraphSAINT [109]. For evaluation, we use prediction accuracy for multi-class and micro-F1 for multi-label datasets.

Results and Analysis. Table 6 and Table 7 summarize the evaluation results of all compared models in semi-supervised and fully-supervised datasets, respectively, under the setting of node classification. We observe that incorporating high-order information plays an important role in improving the performance of models, especially in citation datasets (Cora, CiteSeer, and PubMed) that are of relatively small scale. Most high-order models, such as GRAND, APPNP, and GDC, aim to use graph diffusion matrix $\bar{\boldsymbol{A}} = \sum_{i=0}^{K} \alpha_i \hat{\boldsymbol{A}}^i$ to collect information of distant neighbors. GCNII uses the residual connection with identity mapping to resolve over-smoothing in GNNs and achieves remarkable results. GRAND [26] leverages unlabelled data in semi-supervised settings and obtains state-of-the-art results. As for self-supervised methods, DGI and MVGRL maximize local and global mutual information. MVGRL performs better by replacing the adjacency matrix with graph diffusion matrix but is less scalable. On the contrary, GRACE, which optimizes InfoNCE, does not perform well on citation datasets with the public split.

Table 5: Micro-F1 score (%) of network embedding methods reproduced by CogDL. 50% of nodes are labeled for training in PPI, Blogcatalog, and Wikipedia, 5% in DBLP and Flickr. These datasets correspond to different downstream scenarios.

Rank	Method	PPI (50%)	Wikipedia (50%)	Blogcatalog (50%)	DBLP (5%)	Flickr (5%)	*Reproducible*
1	NetMF [67]	23.73 ± 0.22	**57.42 ± 0.56**	**42.47 ± 0.35**	56.72 ± 0.14	36.27 ± 0.17	*Yes*
2	ProNE [110]	**24.60 ± 0.39**	56.06 ± 0.48	41.16 ± 0.26	56.85 ± 0.28	**36.56 ± 0.11**	*Yes*
3	NetSMF [66]	23.88 ± 0.35	53.81 ± 0.58	40.62 ± 0.35	**59.76 ± 0.41**	35.49 ± 0.07	*Yes*
4	Node2vec [36]	20.67 ± 0.54	54.59 ± 0.51	40.16 ± 0.29	57.36 ± 0.39	36.13 ± 0.13	*Yes*
5	LINE [80]	21.82 ± 0.56	52.46 ± 0.26	38.06 ± 0.39	49.78 ± 0.37	31.61 ± 0.09	*Yes*
6	DeepWalk [63]	20.74 ± 0.40	49.53 ± 0.54	40.48 ± 0.47	57.54 ± 0.32	36.09 ± 0.10	*Yes*
7	SpectralClustering [83]	22.48 ± 0.30	49.35 ± 0.34	41.41 ± 0.34	43.68 ± 0.58	33.09 ± 0.07	*Yes*
8	Hope [61]	21.43 ± 0.32	54.04 ± 0.47	33.99 ± 0.35	56.15 ± 0.22	28.97 ± 0.19	*Yes*
9	GraRep [11]	20.60 ± 0.34	54.37 ± 0.40	33.48 ± 0.30	52.76 ± 0.42	31.83 ± 0.12	*Yes*

Table 6: Accuracy (%) reproduced by CogDL for semi-supervised and self-supervised node classification on Citation datasets. ↓ and ↑ mean our results are lower or higher than the result in original papers.

Method	Cora	Citeseer	Pubmed	*Reproducible*
GRAND [26]	84.8	**75.1**	**82.4**	*Yes*
GCNII [17]	**85.1**	71.3	80.2	*Yes*
MVGRL [39]	83.6 ↓	73.0	80.1	*Partial*
APPNP [49]	84.3 ↑	72.0	80.0	*Yes*
Graph-Unet [31]	83.3 ↓	71.2 ↓	79.0	*Partial*
GDC [50]	82.5	72.1	79.8	*Yes*
GAT [87]	82.9	71.0	78.9	*Yes*
DropEdge [69]	82.1	72.1	79.7	*Yes*
GCN [48]	81.5	71.4 ↑	79.5	*Yes*
DGI [86]	82.0	71.2	76.5	*Yes*
JK-net [99]	81.8	69.5	77.7	*Yes*
Chebyshev [20]	79.0	69.8	68.6	*Yes*

Table 7: Results of node classification on fully-supervised datasets, including full-batch and sampling-based methods. Flickr, Reddit, and ogbn-arxiv use accuracy metric, whereas PPI uses Micro-F1 metric.

	Flickr	PPI	Reddit	arxiv
GCNII	**52.6±0.1**	96.5±0.2	**96.4±0.0**	**72.5±0.1**
GraphSAINT	52.0±0.1	**99.3±0.1**	96.1±0.0	71.5±0.2
GAT	51.9±0.3	97.3±0.2	95.9±0.1	72.3±0.1
Cluster-SAGE	50.9±0.1	97.9±0.1	95.8±0.1	69.8±0.1
GCN	52.4±0.1	75.7±0.1	95.1±0.0	71.7±0.3
APPNP	52.3±0.1	62.3±0.1	96.2±0.1	70.7±0.1
PPRGo	51.1±0.2	48.6±0.1	94.5±0.1	69.2±0.1
SGC	50.2±0.1	47.2±0.1	94.5±0.1	67.1±0.0

4.3 Graph Classification

Setup. Graph classification assigns a label to each graph and aims to map graphs into vector spaces. Graph kernels are historically dominant and employ a kernel function to measure the similarity between pairs of graphs to map graphs into vector spaces with deterministic mapping functions. But they suffer from computational bottlenecks. Recently, graph neural networks attract much attention and indeed show promising results in this task. In the context of graph classification, GNNs often employ the readout operation to obtain a compact representation of the graph and apply classification based on the readout representation. We collect eight popular benchmarks used in graph classification tasks, including Bioinformatics datasets (PROTEINS, MUTAG, PTC, and NCI1), and Social networks (IMDB-B, IMDB-M, COLLAB, REDDIT-B). We implement the following graph classification models and compare their results: GIN [98], DiffPool [105], SAGPool [51], SortPool [111], DGCNN [94] and Infograph [76] are based on GNNs. PATCHY_SAN [60] is inspired by convolutional neural networks. Deep graph kernels (DGK) [100] and graph2vec [58] use graph kernels. As for evaluation, for supervised methods we adopt 10-fold cross-validation with 90%/10% split and repeat 10 times; for unsupervised methods, we perform the 10-fold cross-validation with LIB-SVM [14]. Then we report the accuracy for classification performance of all methods on these datasets.

Results and Analysis. Table 8 reports the results of the aforementioned models on the task, including both unsupervised and supervised graph classification. The development of GNNs for graph classification is mainly in two aspects. One line (like GIN) aims to design more powerful convolution operations to improve the expressiveness. Another line is to develop effective pooling methods to generate the graph representation. Neural network based methods show promising results in bioinformatics datasets (MUTAG, PTC, PROTEINS, and NCI1), where nodes are with given features. But in social networks (IMDB-B, IMDB-M, COLLAB, REDDIT-B) lacking node features, methods based on graph kernels achieve good performance and even surpass neural networks. Graph kernels are more capable of capturing structural information to discriminate non-isomorphic graphs, while GNNs are better encoders with features. Global pooling, which is used in GIN and SortPool, collects node features and applies a readout function. Hierarchical pooling, such as DiffPool and SAGPool, is proposed to capture structural information in different graph levels, including nodes and subgraphs. The experimental results indicate that, though hierarchical pooling seems more complex and intuitively would capture more information, it does not show significant advantages over global pooling.

Table 8: Accuracy Results (%) of both unsupervised and supervised graph classification. ↓ and ↑ mean our results are lower or higher than the results in original papers. ‡ means the experiment is not finished in 24 hours for one seed.

Algorithm	MUTAG	PTC	NCI1	PROTEINS	IMDB-B	IMDB-M	COLLAB	REDDIT-B	*Reproducible*
GIN [98]	**92.06**	**67.82**	**81.66**	75.19	**76.10**	51.80	79.52	83.10 ↓	*Yes*
InfoGraph [76]	88.95	60.74	76.64	73.93	74.50	51.33	79.40	76.55	*Yes*
graph2vec [58]	83.68	54.76 ↓	71.85	73.30	73.90	**52.27**	**85.58** ↑	**91.77**	*Yes*
SortPool [111]	87.25	62.04	73.99 ↑	74.48	75.40	50.47	80.07 ↑	78.15	*Yes*
DiffPool [105]	85.18	58.00	69.09	75.30	72.50	50.50	79.27	81.20	*Yes*
PATCHY_SAN [60]	86.12	61.60	69.82	**75.38**	76.00 ↑	46.40	74.34	60.61	*Yes*
DGCNN [94]	83.33	56.72	65.96	66.75	71.60	49.20	77.45	86.20	*Yes*
SAGPool [51]	71.73 ↓	59.92	72.87	74.03	74.80	51.33	__‡	89.21	*Yes*
DGK [100]	85.58	57.28	__‡	72.59	55.00 ↓	40.40 ↓	__‡	__‡	*Partial*

4.4 Other Graph Tasks

As shown in Table 1, CogDL also provides other important tasks in graph learning. In this section, we briefly introduce the setting and evaluation of some tasks, including heterogeneous node classification, link prediction, multiplex link prediction, etc.

Heterogeneous Node Classification. This task is built for heterogeneous GNNs conducted on heterogeneous graphs such as academic graphs. For heterogeneous node classification, we use macro-F1 to evaluate the performance of all heterogeneous models under the setting of Graph Transformer Networks (GTN) [107]. The heterogeneous datasets include DBLP, ACM, and IMDB. We implement these models, including GTN [107], HAN [92], PTE [79], metapath2vec [21], hin2vec [29].

Link Prediction. Many real applications, such as citation link prediction, can be viewed as a link prediction task [47]. We formulate the link prediction task as: given a part of edges in the graph, after learning representations for each vertex $v \in V$, the method needs to accurately predict the edges that belong to the rest of the edges rather than random edges. In practice, we remove 15 percent of original edges for each dataset and adopt ROC AUC [38] as the evaluation metric.

Multiplex Link Prediction. GATNE [13] formalizes the representation learning problem of multiplex networks and proposes a unified framework to solve the problem in transductive and inductive settings. We follow the setting of GATNE [13] to build the multiplex heterogeneous link prediction task. In CogDL, for those methods that only deal with homogeneous networks, we separate the graphs based on the edge types and train the method on each separate graph. We also adopt ROC AUC as the evaluation metric.

Knowledge Graph Completion. This task aims to predict missing links in a knowledge graph through knowledge graph embedding. In CogDL, we implement two families of knowledge graph embedding algorithms: triplet-based knowledge graph embedding and knowledge-based GNNs. The former includes TransE [9], DistMult [101], ComplEx [84], and RotatE [78]; the latter includes RGCN [72] and CompGCN [85]. We evaluate all implemented algorithms with standard benchmarks including FB15k-237, WN18 and WN18RR with Mean Reciprocal Rank (MRR) metric.

5 DISCUSSIONS

With the success of graph neural networks, there are dozens of graph libraries developed in the past few years, including PGL [1], Graph Nets [5], Jraph [32], Graph-Learn [116], Spektral [35], StellarGraph [19], Deep Graph Library (DGL) [91], and PyTorch Geometric (PyG) [27]. Among them, PyG [27] and DGL [91] are two of the most well-known libraries with efficient operators and easy-to-use APIs. PyG provides CUDA kernels for sparse operators with high data throughput. PyG 2.0 further utilizes GraphGym [106] to achieve new features like reproducible configuration and hyperparameter optimization. DGL not only provides flexible and efficient message-passing APIs but also allows users to easily port and leverage the existing components across multiple deep learning frameworks (e.g., PyTorch [62], TensorFlow [2]). However, CogDL not only supports flexible implementations of GNN models but also provides a unified trainer for the end-to-end training of GNN models with efficient techniques.

6 CONCLUSIONS

This paper introduces CogDL, a comprehensive library for graph deep learning that focuses on the research and applications of graph representation learning. CogDL provides simple APIs such that users can train and evaluate graph learning methods with ease and efficiency. Besides, CogDL implements many state-of-the-art models with wrappers for various graph tasks. Finally, CogDL collects and maintains leaderboards with reproducible configurations on widely-used public datasets. In the future, we will consistently improve the efficiency of CogDL and support more features to satisfy the latest need of both scientific and industrial applications. For example, the dynamic scenario of graph learning, where nodes and edges will continuously be added and removed, needs to be considered. Besides, graph self-supervised learning and pre-training have become one of the most exciting research directions in recent years, which presents a new challenge for graph libraries.

ACKNOWLEDGMENTS

This research was supported by the Natural Science Foundation of China (NSFC) 61825602 and 62276148 and the Tsinghua-Siemens Joint Research Center for Industrial Intelligence and Internet of Things (JCIIOT).

REFERENCES

[1] 2022. PGL. https://github.com/PaddlePaddle/PGL

[2] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2016. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467* (2016).

[3] Sami Abu-El-Haija, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, and Aram Galstyan. 2019. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *ICML'19*. PMLR, 21–29.

[4] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *KDD'19*. 2623–2631.

[5] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018).

[6] Lukas Biewald. 2020. Experiment Tracking with Weights and Biases. https://www.wandb.com/ Software available from wandb.com.

[7] Aleksandar Bojchevski and Stephan Günnemann. 2019. Adversarial attacks on node embeddings via graph poisoning. In *ICML'19*. PMLR, 695–704.

[8] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rózemberczki, Michal Lukasik, and Stephan Günnemann. 2020. Scaling graph neural networks with approximate pagerank. In *KDD'20*.

[9] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. In *NeurIPS'13*.

[10] Bobby-Joe Breitkreutz, Chris Stark, et al. 2008. The BioGRID interaction database. *Nucleic acids research* 36, suppl 1 (2008), D637–D640.

[11] Shaosheng Cao, Wei Lu, and Qiongkai Xu. 2015. Grarep: Learning graph representations with global structural information. In *CIKM'15*. ACM, 891–900.

[12] Shaosheng Cao, Wei Lu, and Qiongkai Xu. 2016. Deep neural networks for learning graph representations. In *AAAI'16*.

[13] Yukuo Cen, Xu Zou, Jianwei Zhang, Hongxia Yang, Jingren Zhou, and Jie Tang. 2019. Representation learning for attributed multiplex heterogeneous network. In *KDD'19*. 1358–1368.

[14] Chih-Chung Chang and Chih-Jen Lin. 2011. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology* 2 (2011), 27:1–27:27. Issue 3. Software available at http://www.csie.ntu.edu.tw/~cjlin/libsvm.

[15] Jinyin Chen, Yangyang Wu, Xuanheng Xu, Yixian Chen, Haibin Zheng, and Qi Xuan. 2018. Fast gradient attack on network embedding. *arXiv preprint arXiv:1809.02797* (2018).

[16] Jianfei Chen, Lianmin Zheng, Zhewei Yao, Dequan Wang, Ion Stoica, Michael W Mahoney, and Joseph E Gonzalez. 2021. ActNN: Reducing Training Memory Footprint via 2-Bit Activation Compressed Training. In *ICML'21*.

[17] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and deep graph convolutional networks. In *ICML'20*.

[18] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *KDD'19*. 257–266.

[19] CSIRO's Data61. 2018. StellarGraph Machine Learning Library. https://github.com/stellargraph/stellargraph.

[20] Michael Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *NeurIPS'16*.

[21] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. 2017. metapath2vec: Scalable representation learning for heterogeneous networks. In *KDD'17*.

[22] Vijay Prakash Dwivedi, Chaitanya K Joshi, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2020. Benchmarking graph neural networks. *arXiv preprint arXiv:2003.00982* (2020).

[23] Negin Entezari, Saba A Al-Sayouri, Amirali Darvishzadeh, and Evangelos E Papalexakis. 2020. All you need is low (rank) defending against adversarial attacks on graphs. In *WSDM'20*. 169–177.

[24] Federico Errica, Marco Podda, Davide Bacciu, and Alessio Micheli. 2020. A fair comparison of graph neural networks for graph classification. In *ICLR'20*.

[25] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *WWW'19*.

[26] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph Random Neural Networks for Semi-Supervised Learning on Graphs. In *NeurIPS'20*.

[27] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).

[28] Fabrizio Frasca, Emanuele Rossi, Davide Eynard, Benjamin Chamberlain, Michael Bronstein, and Federico Monti. 2020. SIGN: Scalable Inception Graph Neural Networks. In *ICML 2020 Workshop on GRL+*.

[29] Tao-yang Fu, Wang-Chien Lee, and Zhen Lei. 2017. Hin2vec: Explore meta-paths in heterogeneous information networks for representation learning. In *CIKM'17*.

[30] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse gpu kernels for deep learning. In *SC'20*. IEEE, 1–14.

[31] Hongyang Gao and Shuiwang Ji. 2019. Graph u-nets. In *ICML'19*.

[32] Jonathan Godwin*, Thomas Keck*, Peter Battaglia, Victor Bapst, Thomas Kipf, Yujia Li, Kimberly Stachenfeld, Petar Veličković, and Alvaro Sanchez-Gonzalez. 2020. *Jraph: A library for graph neural networks in jax.* http://github.com/deepmind/jraph

[33] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and harnessing adversarial examples. In *ICLR'15*.

[34] Marco Gori, Gabriele Monfardini, and Franco Scarselli. 2005. A new model for learning in graph domains. In *IJCNN'05*, Vol. 2. 729–734.

[35] Daniele Grattarola and Cesare Alippi. 2020. Graph neural networks in tensorflow and keras with spektral. *arXiv preprint arXiv:2006.12138* (2020).

[36] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *KDD'16*.

[37] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *NeurIPS'17*. 1025–1035.

[38] James A Hanley and Barbara J McNeil. 1982. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology* 143, 1 (1982).

[39] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *ICML'20*.

[40] Zhenyu Hou, Yukuo Cen, Yuxiao Dong, Jie Zhang, and Jie Tang. 2021. Automated Unsupervised Graph Representation Learning. *IEEE Transactions on Knowledge and Data Engineering* (2021).

[41] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. GraphMAE: Self-Supervised Masked Graph Autoencoders. In *KDD'22*.

[42] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. In *NeurIPS'20*.

[43] Ziniu Hu, Yuxiao Dong, Kuansan Wang, Kai-Wei Chang, and Yizhou Sun. 2020. GPT-GNN: Generative Pre-Training of Graph Neural Networks. In *KDD'20*.

[44] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. 2020. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. In *SC'20*. IEEE Press, Article 72.

[45] Kezhao Huang, Jidong Zhai, Zhen Zheng, Youngmin Yi, and Xipeng Shen. 2021. Understanding and bridging the gaps in current GNN performance optimizations. In *PPoPP'21*. 119–132.

[46] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin R Benson. 2020. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:2010.13993* (2020).

[47] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).

[48] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *ICLR'17*.

[49] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Predict then propagate: Graph neural networks meet personalized pagerank. In *ICLR'19*.

[50] Johannes Klicpera, Stefan Weißenberger, and Stephan Günnemann. 2019. Diffusion Improves Graph Learning. In *NeurIPS'19*.

[51] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. 2019. Self-attention graph pooling. In *ICML'19*.

[52] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. 2021. Training graph neural networks with 1000 layers. In *ICML'21*. PMLR, 6437–6449.

[53] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. 2020. Deepergcn: All you need to train deeper gcns. *arXiv preprint arXiv:2006.07739* (2020).

[54] Xiaoxuan Liu, Lianmin Zheng, Dequan Wang, Yukuo Cen, Weize Chen, Xu Han, Jianfei Chen, Zhiyuan Liu, Jie Tang, Joey Gonzalez, et al. 2022. GACT: Activation Compressed Training for Generic Network Architectures. In *ICML'22*. PMLR.

[55] Qingsong Lv, Ming Ding, Qiang Liu, Yuxiang Chen, Wenzheng Feng, Siming He, Chang Zhou, Jianguo Jiang, Yuxiao Dong, and Jie Tang. 2021. Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks. In *KDD'21*. 1150–1160.

[56] Jianxin Ma, Peng Cui, Kun Kuang, Xin Wang, and Wenwu Zhu. 2019. Disentangled graph convolutional networks. In *ICML'19*. PMLR, 4212–4221.

[57] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards deep learning models resistant to adversarial attacks. In *ICLR'18*.

[58] Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. 2017. graph2vec: Learning distributed representations of graphs. *arXiv preprint arXiv:1707.05005* (2017).

[59] Maxim Naumov, L Chien, Philippe Vandermersch, and Ujval Kapasi. 2010. Cusparse library. In *GPU Technology Conference*.

[60] Mathias Niepert, Mohamed Ahmed, and Konstantin Kutzkov. 2016. Learning convolutional neural networks for graphs. In *ICML'16*.

[61] Mingdong Ou, Peng Cui, Jian Pei, Ziwei Zhang, and Wenwu Zhu. 2016. Asymmetric transitivity preserving graph embedding. In *KDD'16*. 1105–1114.

[62] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS'19*. 8024–8035.

[63] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *KDD'14*. 701–710.

[64] Jinghua Piao, Guozhen Zhang, Fengli Xu, Zhilong Chen, and Yong Li. 2021. Predicting customer value with social relationships via motif-based graph attention networks. In *WWW'21*. 3146–3157.

[65] Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. GCC: Graph contrastive coding for graph neural network pre-training. In *KDD'20*.

[66] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Chi Wang, Kuansan Wang, and Jie Tang. 2019. Netsmf: Large-scale network embedding as sparse matrix factorization. In *WWW'19*.

[67] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. 2018. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec. In *WSDM'18*.

[68] Jiezhong Qiu, Jian Tang, Hao Ma, Yuxiao Dong, Kuansan Wang, and Jie Tang. 2018. Deepinf: Social influence prediction with deep learning. In *KDD'18*.

[69] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2020. Dropedge: Towards deep graph convolutional networks on node classification. In *ICLR'20*.

[70] Aravind Sankar, Junting Wang, Adit Krishnan, and Hari Sundaram. 2020. Beyond localized graph neural networks: An attributed motif regularization framework. In *ICDM'20*. IEEE, 472–481.

[71] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80.

[72] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *European semantic web conference*. Springer, 593–607.

[73] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI magazine* 29, 3 (2008), 93–93.

[74] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).

[75] Chuxiong Sun, Hongming Gu, and Jie Hu. 2021. Scalable and adaptive graph neural networks with self-label-enhanced training. *arXiv preprint arXiv:2104.09376* (2021).

[76] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2020. Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. In *ICLR'20*.

[77] Ke Sun, Zhouchen Lin, and Zhanxing Zhu. 2020. Multi-stage self-supervised learning for graph convolutional networks on graphs with few labeled nodes. In *AAAI'20*, Vol. 34. 5892–5899.

[78] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space. In *ICLR'19*.

[79] Jian Tang, Meng Qu, and Qiaozhu Mei. 2015. Pte: Predictive text embedding through large-scale heterogeneous text networks. In *KDD'15*.

[80] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *WWW'15*.

[81] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2008. Arnetminer: extraction and mining of academic social networks. In *KDD'08*.

[82] Lei Tang and Huan Liu. 2009. Relational learning via latent social dimensions. In *KDD'09*.

[83] Lei Tang and Huan Liu. 2011. Leveraging social media networks for classification. *Data Mining and Knowledge Discovery* 23, 3 (2011), 447–478.

[84] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. Complex embeddings for simple link prediction. In *ICML'16*.

[85] Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. 2020. Composition-based multi-relational graph convolutional networks. In *ICLR'20*.

[86] Petar Velickovic, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax. In *ICLR'19*.

[87] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *ICLR'18*.

[88] Vikas Verma, Meng Qu, Kenji Kawaguchi, Alex Lamb, Yoshua Bengio, Juho Kannala, and Jian Tang. 2021. Graphmix: Improved training of gnns for semi-supervised learning. In *AAAI'21*, Vol. 35. 10024–10032.

[89] Chun Wang, Shirui Pan, Ruiqi Hu, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Attributed graph clustering: A deep attentional embedding approach. *arXiv preprint arXiv:1906.06532* (2019).

[90] Daixin Wang, Peng Cui, and Wenwu Zhu. 2016. Structural deep network embedding. In *KDD'16*. 1225–1234.

[91] Minjie Wang, Lingfan Yu, Da Zheng, Quan Gan, Yu Gai, Zihao Ye, Mufei Li, Jinjing Zhou, Qi Huang, Chao Ma, et al. 2019. Deep graph library: Towards efficient and scalable deep learning on graphs. *arXiv preprint arXiv:1909.01315* (2019).

[92] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *WWW'19*. 2022–2032.

[93] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. 2021. GNNAdvisor: An Efficient Runtime System for GNN Acceleration on GPUs. In *OSDI'21*.

[94] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. 2019. Dynamic graph cnn for learning on point clouds. *Acm Transactions On Graphics (tog)* 38, 5 (2019), 1–12.

[95] Marcin Waniek, Tomasz P Michalak, Michael J Wooldridge, and Talal Rahwan. 2018. Hiding individuals and communities in a social network. *Nature Human Behaviour* 2, 2 (2018), 139–147.

[96] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying Graph Convolutional Networks. In *ICML'19*.

[97] Jiarong Xu, Yizhou Sun, Xin Jiang, Yanhao Wang, Chunping Wang, Jiangang Lu, and Yang Yang. 2022. Blindfolded Attackers Still Threatening: Strict Black-Box Adversarial Attacks on Graphs. In *AAAI'22*.

[98] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks?. In *ICLR'19*.

[99] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. In *ICML'18*.

[100] Pinar Yanardag and SVN Vishwanathan. 2015. Deep graph kernels. In *KDD'15*.

[101] B. Yang, Wen tau Yih, X. He, Jianfeng Gao, and L. Deng. 2015. Embedding Entities and Relations for Learning and Inference in Knowledge Bases. In *ICLR'15*.

[102] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *ICML'16*. PMLR, 40–48.

[103] Zhiju Yang, Weiping Pei, Monchu Chen, and Chuan Yue. 2022. WTAGRAPH: Web Tracking and Advertising Detection using Graph Neural Networks. In *IEEE Symposium on Security and Privacy*.

[104] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD'18*. 974–983.

[105] Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. 2018. Hierarchical graph representation learning with differentiable pooling. In *NeurIPS'18*.

[106] Jiaxuan You, Zhitao Ying, and Jure Leskovec. 2020. Design space for graph neural networks. In *NeurIPS'20*.

[107] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J Kim. 2019. Graph transformer networks. In *NeurIPS'19*.

[108] Reza Zafarani and Huan Liu. 2009. Social computing data repository at ASU.

[109] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2020. GraphSAINT: Graph Sampling Based Inductive Learning Method. In *ICLR'20*.

[110] Jie Zhang, Yuxiao Dong, Yan Wang, Jie Tang, and Ming Ding. 2019. ProNE: fast and scalable network representation learning. In *IJCAI'19*. 4278–4284.

[111] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An end-to-end deep learning architecture for graph classification. In *AAAI'18*.

[112] Xiaotong Zhang, Han Liu, Qimai Li, and Xiao-Ming Wu. 2019. Attributed graph clustering via adaptive graph convolution. In *IJCAI'19*.

[113] Xiang Zhang and Marinka Zitnik. 2020. Gnnguard: Defending graph neural networks against adversarial attacks. In *NeurIPS'20*, Vol. 33. 9263–9275.

[114] Qinkai Zheng, Yixiao Fei, Yanhao Li, Qingmin Liu, Minhao Hu, and Qibo Sun. 2020. KDD CUP 2020 ML Track 2 Adversarial Attacks and Defense on Academic Graph 1st Place Solution.

[115] Dingyuan Zhu, Ziwei Zhang, Peng Cui, and Wenwu Zhu. 2019. Robust graph convolutional networks against adversarial attacks. In *KDD'19*. 1399–1407.

[116] Rong Zhu, Kun Zhao, Hongxia Yang, Wei Lin, Chang Zhou, Baole Ai, Yong Li, and Jingren Zhou. 2019. AliGraph: a comprehensive graph neural network platform. *Proceedings of the VLDB Endowment* 12, 12 (2019), 2094–2105.

[117] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. *arXiv preprint arXiv:2006.04131* (2020).

[118] Xu Zou, Qinkai Zheng, Yuxiao Dong, Xinyu Guan, Evgeny Kharlamov, Jialiang Lu, and Jie Tang. 2021. TDGIA: Effective injection attacks on graph neural networks. In *KDD'21*. 2461–2471.

[119] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. 2018. Adversarial attacks on neural networks for graph data. In *KDD'18*. 2847–2856.

A COGDL PACKAGE

In this section, we introduce the details of the CogDL library.

A.1 Basic Components

The basic elements in CogDL include `Graph`, `Dataset`, and `Model`.

Graph. The `Graph` is the basic data structure of CogDL to store graph data with abundant graph operations. `Graph` supports both convenient graph modification, such as removing/adding edges, and efficient computing, such as SpMM. We also provide common graph manipulations, including adding self-loops, graph normalization, sampling neighbors, obtaining induced subgraphs, etc.

Dataset. The `dataset` component reads in data and processes it to produce tensors of appropriate types. Each dataset specifies its loss function and evaluator through function `get_loss_fn` and `get_evaluator` for training and evaluation. Our package provides many real-world datasets and easy-to-use interfaces for users to define customized datasets. CogDL provides two ways to construct a customized dataset. One way is to convert raw data files into the required format of CogDL and specify the argument *data_path*. The other way is to define a customized dataset class with the necessary functions to load and process the data.

Listing 3: Definition of datasets

```
class CoraDataset(Dataset):
  def process(self):
    """preprocess data to cogdl.Graph"""
  def get_evaluator(self):
    """Return evaluator defined in CogDL"""
    return accuracy
  def get_loss_fn(self):
    """Return loss function defined in CogDL"""
    return cross_entropy_loss
```

Model. CogDL implements various graph neural networks as research baselines and for applications. A GNN layer such as `GCNLayer` has already been implemented based on `Graph` and efficient sparse operators. A `model` in the package consists of the model builder (i.e., `__init__`) and forward propagation (i.e., `forward`) functions in the PyTorch style. Listing 4 shows the APIs implemented in each model to provide a unified paradigm for usage, where `add_args` is used to define model-specific hyperparameters for experiments.

Listing 4: Definition of GNN models

```
from cogdl.layers imoprt GCNLayer

class GCN(BaseModel):
  def add_args(parser):
    parser.add_argument("--hidden", type=int)

  def __init__(self, nfeats, nclasses, hidden):
    self.layer1 = GCNLayer(nfeats, hidden)
    self.layer2 = GCNLayer(hidden, nclasses)

  def forward(self, graph):
    h = graph.x
    h = self.layer1(graph, h)
    h = self.layer2(graph, h)
    return h
```

A.2 Basic Usage

Figure 2 illustrates the benefits of CogDL's unified trainer and modular wrappers over PyG and DGL. Users can directly use the `Trainer` for running the experiments and do not need to write tedious code to build the training loop for GNN training and evaluation. Besides, we introduce another easy-to-use usage for conducting experiments through `experiment` API. We can feed a dataset, a model, and hyper-parameters to the `experiment` API, which calls the `Trainer` API. Producing the results with one-line code provides a convenient way for experiments. Furthermore, we integrate a popular library, Optuna [4], into CogDL to enable hyper-parameter search. By feeding the `search_space` function that defines the search space of hyper-parameters, CogDL will start the searching process for the validation metric and output the best results.

Listing 5: Basic usages of CogDL

```
from cogdl import experiment
experiment(dataset="cora", model="gcn", hidden_size=32, max_epoch
    =200) # basic usage

def func(trial):
  return {
    "hidden_size": trial.suggest_categorical("hidden_size", [32, 64,
      128]),
    "dropout": trial.suggest_uniform("dropout", 0.1, 0.8),
  }
experiment(dataset="cora", model="gcn", search_space=func) # hyper-
    parameter search
```

B APPLICATIONS

Our library has been successfully applied to a large-scale academic mining and search system—AMiner (https://www.aminer.cn/)—for the analysis of papers and scholars, as illustrated in Figure 3. There are more than 320 million papers with more than 1 billion citation links in the AMiner database. Most applications (e.g., paper tagging) in AMiner are related to academic papers and scholars. Therefore, it is important to obtain accurate embeddings for papers and scholars for downstream tasks.

Take paper tagging as an example. Each publication in AMiner has several tags extracted from the raw text (e.g., title and abstract) of each publication. However, publications with citation links may have similar tags, and we can utilize the citation network to improve the quality of tags. The paper tagging problem can be considered as a multi-label node classification task, where each label represents a tag. We can utilize paper embeddings to improve the quality of paper tags since we find many papers missing tags in the AMiner database. With the help of the AMiner team, we train a downstream classifier for the paper tagging task on 4.8 million papers in computer science. The experimental results demonstrate that those embeddings could help improve the recall by 12.8% compared with the previous method. In addition to the paper tagging task, the embeddings generated by CogDL can also help the paper recommendation of AMiner. Specifically, the AMiner homepage gives paper recommendations for users based on users' historical behaviors. The quality of the paper embeddings plays an important role in the recommendation. We hope that our library can help more users to build better services, especially those web-related applications, benefiting the community of graph learning.

CogDL

```
model = GCN(...)
data = CoraDataset()[0]
mw = ModelWrapper(model, ...)
dw = DataWrapper(data, ...)
trainer = Trainer(epochs=100)
result = trainer.run(mw, dw)
```

PyG

```
model = GCN(...)
data = Planetoid(name="Cora")[0]
optimizer = torch.optim.Adam(...)
for epoch in range(100):
    pred = model(data.x, data.edge_index)
    labels = data.y
    mask = data.train_mask
    loss = F.nll_loss(pred[mask],
                      labels[mask])
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    val_acc = evaluate(model, data)
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model = deepcopy(model)
result = test(best_model, data)
```

DGL

```
model = GCN(...)
data = CoraGraphDataset()[0]
optimizer = torch.optim.Adam(...)
for epoch in range(100):
    pred = model(data)
    labels = data.ndata['label']
    mask = data.ndata['train_mask']
    loss = F.nll_loss(pred[mask],
                      labels[mask])
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    val_acc = evaluate(model, data)
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model = deepcopy(model)
result = test(best_model, data)
```

Figure 2: Illustration of training the GCN model on the Cora dataset using CogDL, PyG, and DGL. The trainer of CogDL takes over the entire loop of the training and evaluation process. The ModelWrapper consists of training and testing steps, while the DataWrapper handles the data preparation and transformation.

Table 9: Dataset statistics for node classification ("m" stands for multi-label classification)

Setting	Dataset	#Nodes	#Edges	#Features	#Classes	# Train / Val / Test
Semi-supervised Setting	Cora	2,708	5,429	1,433	7	140 / 500 / 1,000
	Citeseer	3,327	4,732	3,703	6	120 / 500 / 1,000
	Pubmed	19,717	44,338	500	3	60 / 500 / 1,000
Fully-Supervised Setting	PPI	56,944	818,736	50	121 (m)	0.79 / 0.11 / 0.10
	Flickr	89,350	899,756	500	7	0.50 / 0.25 / 0.25
	Reddit	232,965	11,606,919	602	41	0.66 / 0.10 / 0.24
	Yelp	716,847	6,977,410	300	100 (m)	0.75 / 0.10 / 0.15
	ogbn-arxiv	169,343	1,166,243	128	40	0.54 / 0.18 / 0.28
Network Embedding Setting	PPI	3,890	76,584	-	50 (m)	0.50 / - / 0.50
	Wikipedia	4,777	184,812	-	40 (m)	0.50 / - / 0.50
	Blogcatalog	10,312	333,983	-	39 (m)	0.50 / - / 0.50
	DBLP	51,264	127,968	-	60 (m)	0.05 / - / 0.95
	Flickr	80,513	5,899,882	-	195 (m)	0.05 / - / 0.95

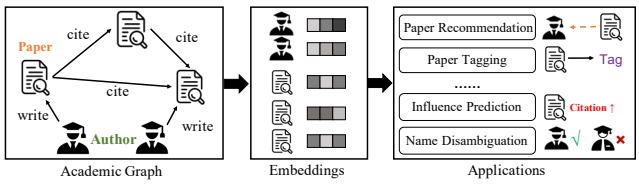


Figure 3: Illustration of how CogDL helps real-world applications in AMiner. Most applications of AMiner rely on a large-scale academic graph. CogDL can help AMiner to generate accurate embeddings for papers and scholars from the billion-scale academic graph for downstream tasks.

Table 10: Dataset statistics for graph classification

Dataset	#Graphs	#Classes	#Feats	#Nodes	#Edges
MUTAG	188	2	7	17.9	19.8
PTC	344	2	18	14.3	14.7
PROTEINS	1,113	2	3	39.1	72.8
NCI1	4,110	2	37	29.8	32.3
IMDB-B	1,000	2	-	19.8	96.5
IMDB-M	1,500	3	-	13.0	65.9
REDDIT-B	2,000	2	-	429.6	497.8
COLLAB	5,000	3	-	74.5	2457.8